COMPARING NUMERICAL SOLUTIONS TO THE NAVIER-STOKES LID DRIVEN CAVITY PROBLEM

CLAIRE CURRY

1. Introduction. The Navier-Stokes equations are the premiere continuum model of fluid flow ranging from oceanic currents, to atmospheric flow and incidental breezes through the office on a warm summer day. Even where models have been specialized to particular applications to focus on particular scales of motion in time or in space, weather, glaciers, porous media, these are often but facets of the underlying Navier-Stokes, cut by the modeler for its particular shine. This project takes an eye to the gem that is Navier-Stokes, for the exercise of grasping fluid dynamics by the lapels and demanding answers.

Several discretizations strategies are used for simulations and benchmarking to make the bulk of a comparative study, though perhaps unfortunately for the precise topic of this course a large portion of the adversity overcome to present these results are of a computer coding nature. When particularly salient to the comparison of methods these adversities may be commented on.

2. Problem Statement.

2.1. The Model. There are a number of ways to denote the Navier-Stokes equation, but ours will be split into four terms such that each is handled differently by our numerical techniques. First as definitions take $\vec{u} = (u(x,y,t), v(x,y,t))$ as our velocity field representing fluid flow, $\rho = \rho_0$ as a constant density parameter, ν as a the kinetic viscosity of the fluid, and $p(x,y,t)$ as a pressure field. We can then write the Incompressible Navier-Stokes equations.

$$\text{(Momentum)} \qquad \frac{\partial \vec{u}}{\partial t} = -(\vec{u}\cdot\nabla)\vec{u} - \frac{1}{\rho_0}\nabla p + \nu\Delta\vec{u}$$

$$\text{(Continuity)} \qquad \nabla\cdot\vec{u} = 0$$

And to be explicit with notation, it should be stated that in two dimensions.

$$(\vec{u}\cdot\nabla)\vec{u} = \begin{bmatrix} u\partial_x u + v\partial_y u \\ u\partial_x v + v\partial_y v \end{bmatrix} \qquad \Delta\vec{u} = \begin{bmatrix} \Delta u \\ \Delta v \end{bmatrix}$$

For the purpose of this numerical exercise we will eschew units to focus on the unitless formulation. This requires the definition of unitless versions of each of our solution functions and variables, the resulting system can be written:

$$\text{(Momentum)} \qquad \frac{\partial \vec{u}}{\partial t} = -(\vec{u}\cdot\nabla)\vec{u} - \nabla p + \frac{1}{Re}\Delta\vec{u}$$

$$\text{(Continuity)} \qquad \nabla\cdot\vec{u} = 0$$

where Re is the Reynolds number quantifying the relative contributions of diffusion and convection of the flow. A high Reynolds number implies that convective or *inertial* forces dominate, e.g; river swimming, and summer breezes, and a lower Reynolds number would correspond to diffusive forces dominating.

This manuscript is for review purposes only.

2.2. The Problem. There is a well used benchmark for the Navier-Stokes equations referred to as the "Lid-Driven Cavity" (LDC) problem where the accuracy and consistency of numerical methods can be compared to the thorough and professional work of other computational fluid dynamicist. For this reason we have chosen the LDC problem for this project. We can state it thusly:

Find $\vec{u}(x, y, t)$ and $p(x, y, t)$ which solve the Momentum and Continuity equations on the interior of the rectangular domain $\Omega = [0, L_x] \times [0, L_y] \times [0, T]$. We impose no-slip boundary conditions on three of four edges of the domain

$$\vec{u}(0, y) = \vec{u}(L_x, y) = 0$$
$$\vec{u}(x, 0) = 0$$

and one *driving* boundary on the *lid*

$$\vec{u}(x, L_y) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The physical scenario this problem models being a rectangular chamber of fluid (infinitely long in one dimension to similar two dimensional flow) with a conveyor belt as its lid in contact with the fluid and driven at a constant speed. This problem is known to create vortical flow around the container, and if accurately computed, it can even demonstrate smaller opposing vortexes in the lower corners.

3. Numerical Methods. We will be numerically solving these equations via a variety of finite difference discretizations on cell centered grid points. We will be applying these discretizations term wise.

To define our variables we will be writing our discretization as $u(x_i, y_j, t_n) = U^n_{i,j}$ where $x_i = i\Delta x$, $y_j = j\Delta y$ and $t_n = n\Delta t$ similiarly for $v(x_i, y_j, t_n) = V^n_{i,j}$. We will be solving on an $N_x \times N_y$ grid of interior points corresponding to $i = 1, 2, \ldots, N_x$ and $j = 1, 2, \ldots, N_y$.

3.1. Convection. The first term we will discuss is the convection term $-(\vec{u}\cdot\nabla)\vec{u}$, a non-linear version of the standard linear transport term $\vec{a} \cdot \nabla u$ for constant $\vec{a}$. Our methods for discretizing the non-linear case will be similar to those commonly used for the linear case.

3.1.1. Up-winding. In the linear case up-winding is first order accurate and it's lightweight to implement. We define the up-winding discretization:

$$\begin{aligned}
(\vec{u} \cdot \nabla)\vec{u} = &\begin{cases} U_{i,j}\left(\frac{\vec{U}_{i,j} - \vec{U}_{i-1,j}}{\Delta x}\right) & \text{if}, U_{i,j} \geq 0 \\ U_{i,j}\left(\frac{\vec{U}_{i+1,j} - \vec{U}_{i,j}}{\Delta x}\right) & \text{if}, U_{i,j} < 0 \end{cases} \\
+ &\begin{cases} V_{i,j}\left(\frac{\vec{U}_{i,j} - \vec{U}_{i-1,j}}{\Delta x}\right) & \text{if}, V_{i,j} \geq 0 \\ V_{i,j}\left(\frac{\vec{U}_{i+1,j} - \vec{U}_{i,j}}{\Delta x}\right) & \text{if}, V_{i,j} < 0 \end{cases} \\
&+ O(\Delta x)
\end{aligned}$$

3.1.2. Centered Difference. A poor choice of spacial discretizations for the pure advection equations due to abysmal stability, however the presence of the diffusion term to be included in the full Navier-Stokes dramatically changes centered

stability making it an attractive choice for a narrow stencil with second order accuracy. We write this discretization as

$$(\vec{u}\cdot\nabla)\vec{u} = U_{i,j}\left(\frac{\vec{U}_{i+1,j}-\vec{U}_{i-1,j}}{2\Delta x}\right) + V_{i,j}\left(\frac{\vec{U}_{i,j+1}-\vec{U}_{i,j-1}}{2\Delta y}\right) + O(\Delta x^2)$$

3.2. Diffusion. Our discretizations of the diffusion term is the ever popular five-point Laplacian.

$$\Delta u = \left(\frac{\vec{U}_{i+1,j}-2\vec{U}_{i,j}+\vec{U}_{i-1,j}}{\Delta x^2} + \frac{\vec{U}_{i,j+1}-2\vec{U}_{i,j}+\vec{U}_{i,j-1}}{\Delta y^2}\right)$$

The severe stability requirements of this discretization will be handled by implicit time stepping for this term.

3.3. Time Discretization. We employ two time stepping methods for this problem. The first is a mixed implicit explicit scheme where the convection term is handled explicitly and the diffusion term handled implicitly. We can write this scheme with some suppressed notation

$$\frac{\vec{U}^{n+1}-\vec{U}^n}{\Delta t} + O(\Delta t) = -U^n D_x\vec{U}^n - V^n D_y\vec{U}^n + \frac{1}{Re}\left(D_x^2\vec{U}^{n+1} + D_y^2\vec{U}^{n+1}\right)$$

where $D_x^{(k)}$,$D_y^{(k)}$ are general discrete kth derivative operators.

The second time discretization is the Alternating-Direction-Implicit (ADI) discretization. The underlying logic is to split the operator into two half time steps each handling the x derivatives and y derivatives seperately to create two tri-diagonal banded solves. In theory this should provide a significant benefit in computation time, however time constraints and Python library constraints has prevented us from properly implementing the efficient banded solves. As such we have been unable to reap any benefits of this structure save for achieving $O(\Delta t^2)$ Crank-Nicholson-like accuracy without requiring any truly nonlinear solves.

$$\frac{\vec{U}^{*}-\vec{U}^n}{2\Delta t} = -U^n D_x\vec{U}^{*} - V^n D_y\vec{U}^n + \frac{1}{Re}\left(D_x^2\vec{U}^{*} + D_y^2\vec{U}^n\right)$$

$$\frac{\vec{U}^{n+1}-\vec{U}^{*}}{2\Delta t} = -U^{*} D_x\vec{U}^{*} - V^{*} D_y\vec{U}^{n+1} + \frac{1}{Re}\left(D_x^2\vec{U}^{*} + D_y^2\vec{U}^{n+1}\right) + O(\Delta t^2)$$

We feel compelled to mention that this was the height of ambition for this project and that the author takes great pride in successfully scripting this ADI discretization for a relatively complex nonlinear system.

A final note, due to this implementational complexity, we have provided code only for centered differencing of the convection term with ADI.

3.4. Pressure Correction. A key difficulty in numerically integrating the Navier-Stokes equations is the problematic pairing of the pressure $p(x, y, t)$ field and the zero divergence continuity equation. While with constant density we have three equations and three unknowns, how exactly we can utilize the zero divergence criteria to determine the proper pressure is a non-trivial exercise.

Here we rely on the wisdom of the ancients and utilize Chorin's (1) operator splitting algorithm first proposed in 1968. Simply stated the logic goes that "pressure", or rather a pressure related scalar, is calculated as a velocity correction which enforces zero divergence.

Algorithm 3.1 Chorin's Project Method

1. Calculate a timestep without contribution from pressure (Example Discretization)
$$\frac{\vec{U}^* - \vec{U}^n}{\Delta t} = -UD_x\vec{U} - VD_y\vec{U} + \frac{1}{Re}\left(D_x^2\vec{U} + D_y^2\vec{U}\right).$$
2. Caluate a pressure correction by solving the pressure Poisson problem:
$$-\Delta p = \frac{1}{\Delta t}\nabla\cdot\vec{u} \approx \frac{1}{\Delta t}\left(D_x\vec{U}^* + D_y\vec{U}^*\right) \qquad \frac{\partial p}{\partial \hat{n}}|_{\partial\Omega} = 0.$$
3. Correct velocities
$$\vec{U}^{n+1} = \vec{U}^* + \Delta t\nabla p.$$

3.5. Pressure Poisson Solve. Finally we discuss the problem of the pressure Poisson solve. Similar to the diffusion term in the Navier-Stokes we will be discretizing with the use of the 5-point Laplacian.

$$-\Delta p = f \Rightarrow -\left(\frac{P_{i-1,j} - 2P_{i,j} + P_{i+1,j}}{\Delta x^2}\right) - \left(\frac{P_{i,j-1} - 2P_{i,j} + P_{i,j+1}}{\Delta y^2}\right) = f_{i,j}$$

and enforcing boundary conditions using a centered difference

$$\frac{P_{i+1,j} - P_{i-1,j}}{2\Delta x} = 0 \Rightarrow P_{i+1,j} = P_{i-1,j}$$
$$\frac{P_{i,j+1} - P_{i,j-1}}{2\Delta y} = 0 \Rightarrow P_{i,j+1} = P_{i,j-1}.$$

As example we can write the bottom left corner explicitly

$$-\left(\frac{-2P_{1,1} + 2P_{2,1}}{\Delta x^2}\right) - \left(\frac{-2P_{1,1} + 2P_{1,2}}{\Delta y^2}\right) = f_{1,1}.$$

This produces a singular matrix with an eigenvector associated with a zero eigenvalue. This eigenvector can be written as the vector of all ones. $\vec{w}_0 = (1, 1 \ldots, 1, 1)^T$. Worse yet, due to floating point error, when represented numerically this matrix is only *numerically singular* presenting a spurious non-zero eigenvalue at the scale of round off error. This requires special care to avoid nonsense solutions. For this project the particular correction employed is to satisfy the solvability condition of the exact linear system by modification of the forcing $\vec{f}$ vector to remove the arithmetic mean.

$$\vec{f}^* = \vec{f} - \frac{\sum_{i=1}^{N_x}\sum_{j=1}^{N_y} f_{i,j}}{N_x N_y}$$

This correction is performed somewhat ad-hoc, however the effectiveness will be verified as part of our benchmarking process.

4. Verification of Numerical Methods. This problem has no known exact solutions and thus we rely on the efforts of previous computational fluid dynamicists for comparison. In particular we rely on the work of Ghia et al. (2) who provides high-resolution readings from the geometric center lines $x = 0.5$ and $y = 0.5$ for the steady-state solution of the LDC problem to which we can compare our results.

Through experimentation on our Navier-Stokes code we've found that larger Reynolds number dramatically increases the time required to reach steady state, this and the fine spacial discretizations required for accuracte results, we have been computationally limited in the size of Reynolds number we can accurately match to Ghia's benchmarks. However, our best results for $Re = 100, 400$ are shown in figure 1.

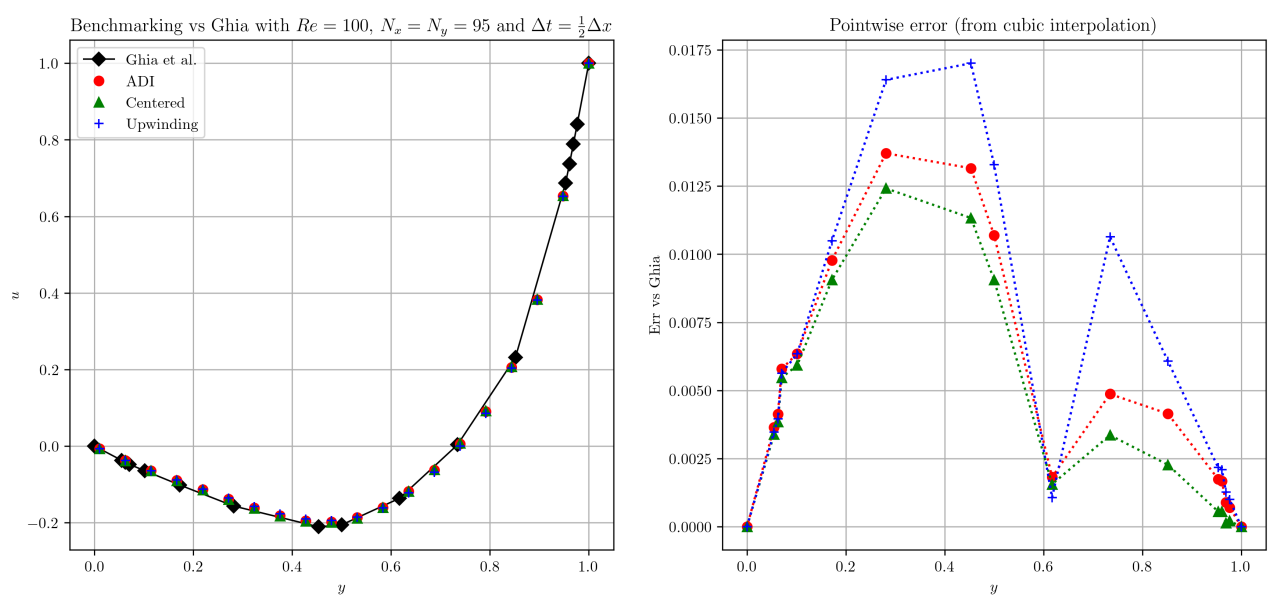
We observe good agreement between our results and that of Ghia et al. For $Re = 100$ we can see the relative effectiveness of our various discretizations. Not surprisingly our only first order in space up-winding method is the least accurate however, not to the degree we may have expected. In addition we may be surprised to note that ADI is only second best to centered differencing. We suspect this is due to our benchmarking on the steady state which likely dilutes the value of being second order in time.

For $Re = 400$ we see decent agreement with benchmarks, however our results are certainly deteriorating somewhat despite the increase in spacial resolution from $N_x = N_y = 95$ to $N_x = N_y = 135$. In this test however we do begin to see a clearer differential between our second order in space methods vs up-winding.

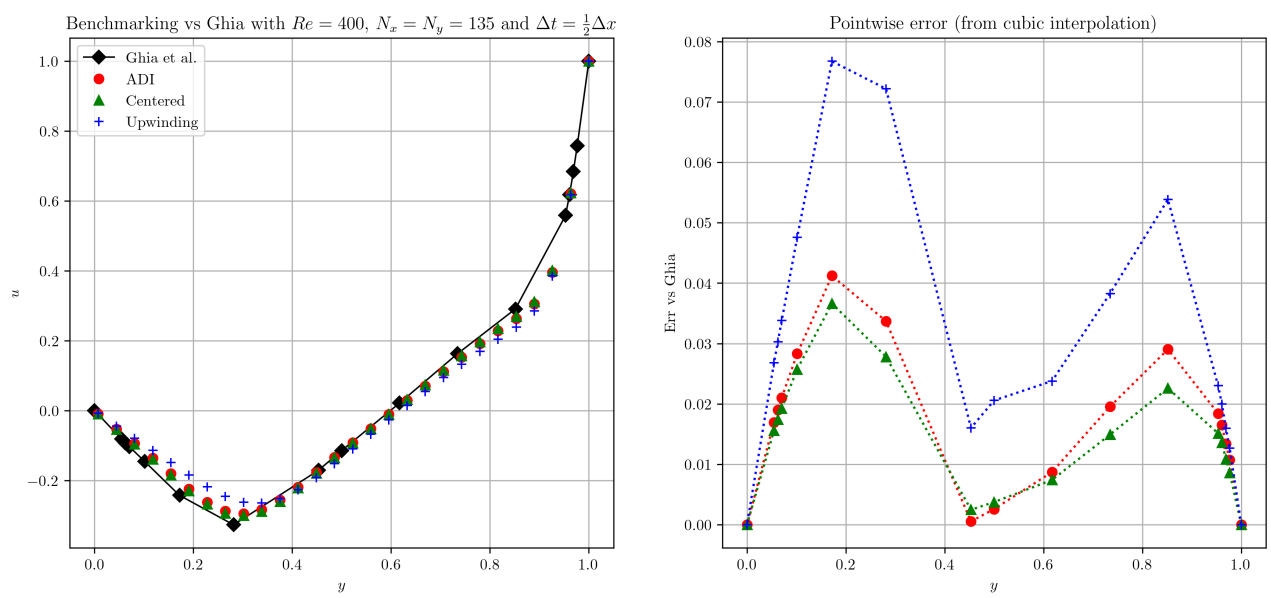
As promised we can additionally benchmark our Poisson problem code (Figure 2). The results of which unfortunately leave much to be desired, in particular we see first order errors in each of the four corners of our Poisson solve. This is especially concerning as the largest locations of divergence seen in our Chorin algorithm occur in the top right and top left corners where the forcing boundary condition occurs. Despite much time, a fix for this has not been found. We recommend in future that a fundamentally different tact is used to approach the Poisson problem, either perhaps a edge centered finite difference rather than cell centered, or spectral method for high accuracy. Despite these short comings, we have not been prevented in achieving modest results from our numerical solutions.

Finally we are curious regarding the relative speed of computation for each of our methods. Relevant to this section is a description of the code all of which has been written in Python with heavy usage of the NumPy and SciPy libraries. NumPy for manipulation of matrices and SciPy in particular for its implementation of sparse matrix solvers.

Each discretization strategy will be ran on the LDC problem for $t \in [0, 1]$ with increasing numbers of grid points. Additionally we will enforce the relation $\Delta x = \Delta t$ for simplicity with $N_x = N_y = N$. Figure 3 shows the results of our tests and the inefficiency of our code is unfortunately laid bare. The most expensive compontent of the algorithm is of course the linear solve, even done through SciPy's sparse matrix library, it appears that computation costs grow quadratically (albeit with a very low starting cost). The SciPy sparse solver was bench marked on our previously used

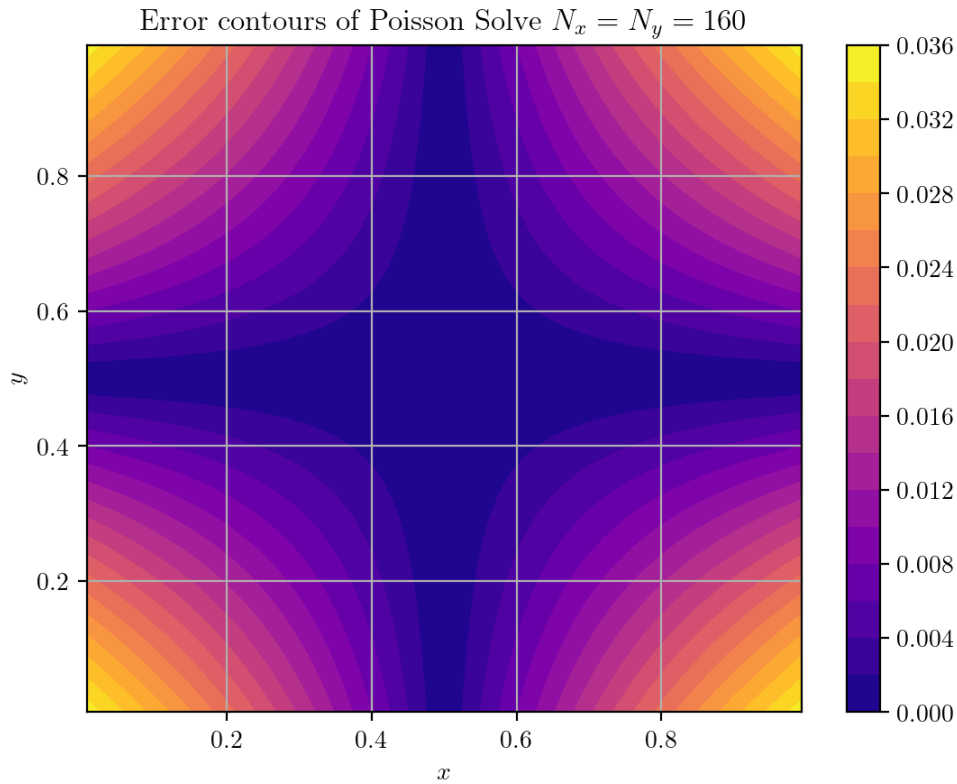


(a) Convergence to the centerline $u(0.5, y)$ for the steady state LDC problem at $Re = 100$

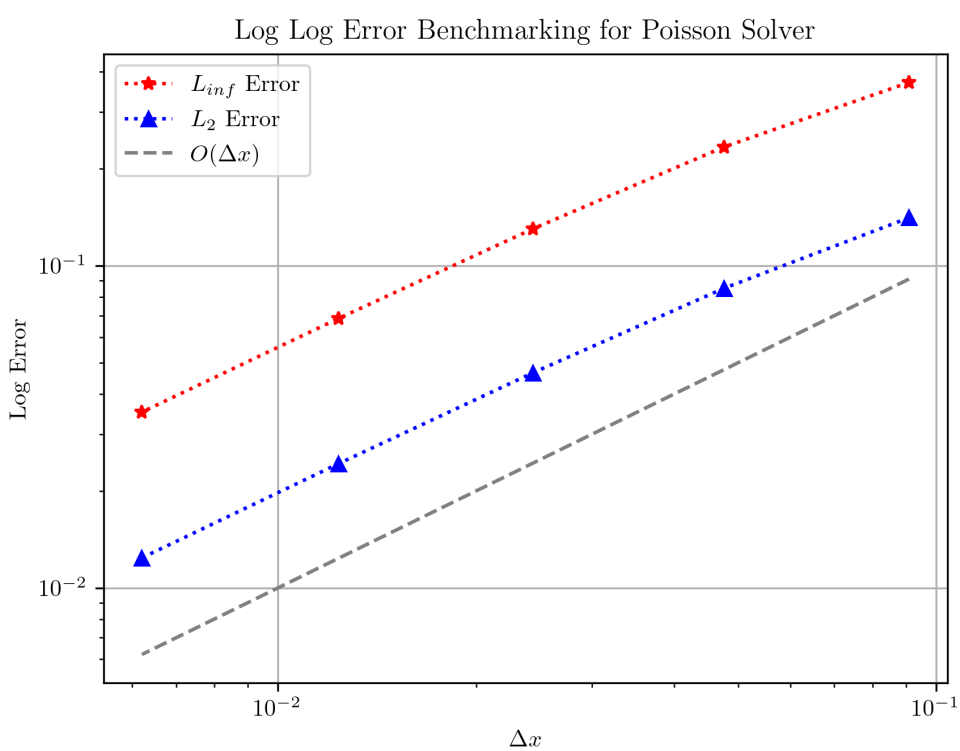


(b) Convergence to the centerline $u(0.5, y)$ for the steady state LDC problem at $Re = 400$

Fig. 1: Verification of numerical methods on the LDC problem



(a) Error contours for the Poisson problem $\Delta u = -2\pi \cos(\pi x)\cos(\pi y)$ solved numerically



(b) Log Log convergence plot of the numerical Poisson solve for refinement of discretizations.

Fig. 2: Verification of numerical methods on the pressure Poisson problem

This manuscript is for review purposes only.

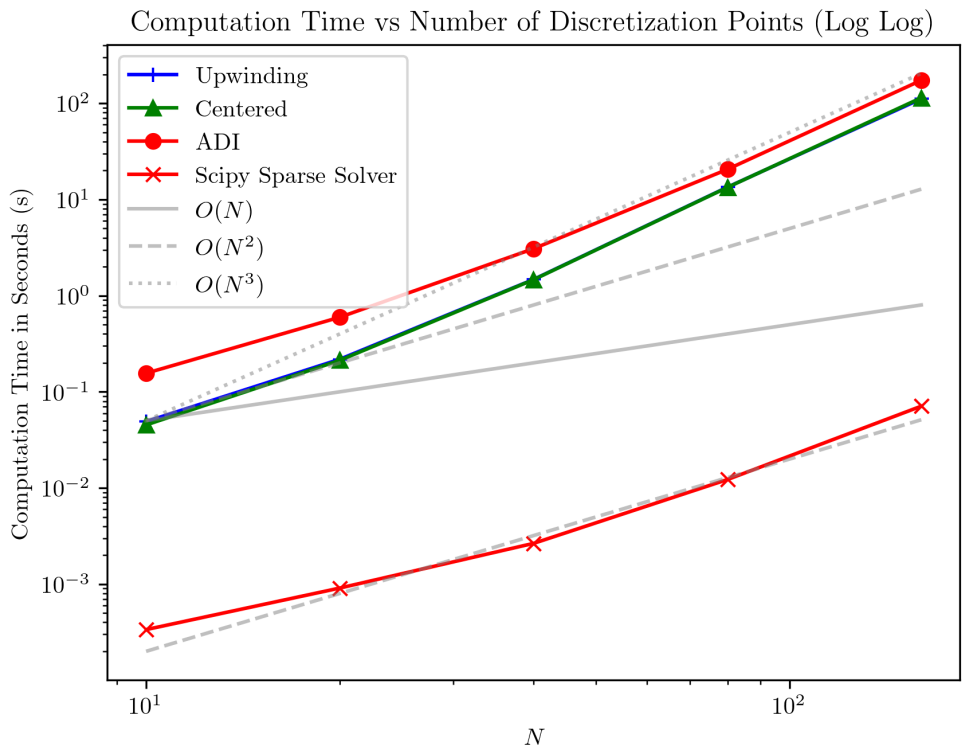


Fig. 3: Computation time vs grid points

Poisson problem representing a characteristic matrix structure seen in the Navier-Stokes discretization. This combined with the reducing size of the timestep required to retain accuracy leading to a painful $O(N^3)$ computation cost. Certainly this tells us that a better Navier-Stokes solver would put significantly more work into boosting efficiency than was afforded in this project.

5. The Pretty Pictures. As this project was born out of an enthusiasm for fluid dynamics we would amiss not to see the sights as it were, now that we have already made the trip. In particular Ghia (2) shows the formation of smaller sub vortices (Figure 4) in the bottom right and bottom left corner of the domain for the mild $Re = 400$ case which we have successfully benchmarked. We will attempt to observe these for ourselves. Our testing of various discretizations has led us to the conclusion that mixed explicit implicit with centered differencing is the best balance of computational efficiency and accuracy for our purposes.

We will recompute the steady state on an ambitiously fine grid $N_x = N_y = 150$ for $Re = 400$ and calculate the stream-function ψ by the formula

$$\Delta\psi = -\left(\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}\right)$$

requiring an additional Poisson solve. The results are shown in figure 5 which

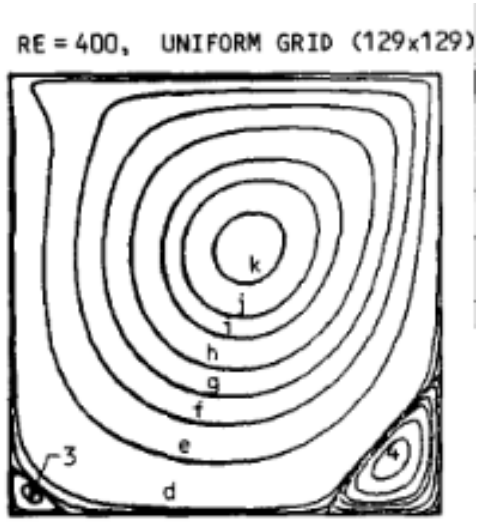


Fig. 4: Figure from Ghia et al. (2) showing sub vortices

we have styled to mimic the presentation from Ghia. As we may expect given our successful benchmarking, we are indeed capable of recreating the Ghia figure to a high degree of subjective accuracy. Indeed, the precise contour levels plotted in figure 5 are pulled from the Ghia article. The feeling is like staring entranced into a windowed washing machine.

6. Conclusion. The Naiver-Stokes is a wonderful and vibrant system whose most common benchmark has with in it vortices, vortices and opposing vortices. Due to nonlinearity and analytic complexity we turn to numerical schemes to solve this system only to find that the numerical perspective has its own challenges of coupling pressure and zero divergence, Poisson solves inside time stepping and breaking and recombining the differential operator to avoid nonlinear solves. The Danish mathematician Piet Hein once said "Problems worthy of attack, prove their worth by fighting back," and that has been true of our experience in tackling this numerical problem.

References.

[1] A. J. Chorin, *Numerical solution of the navier-stokes equations*, Mathematics of computation, 22 (1968), pp. 745–762.

[2] U. Ghia, K. Ghia, and C. Shin, *High-re solutions for incompressible flow using the navier-stokes equations and a multigrid method*, Journal of Computational Physics, 48 (1982), pp. 387–411, https://doi.org/https://doi.org/10.1016/0021-9991(82)90058-4, https://www.sciencedirect.com/science/article/pii/0021999182900584.

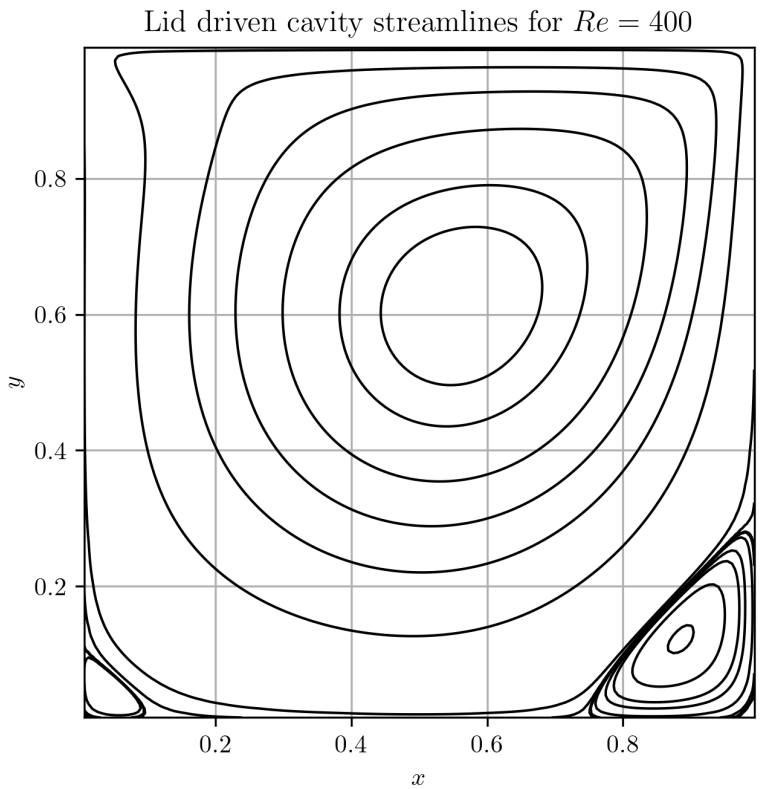


Fig. 5: Opposing vorticies created by the lid driven cavity problem